

Exercícios: Desafios Integrados de Lógica

Turma: LionsDev

Tópicos: Revisão dos fundamentos de programação.

⚠ **Aviso:** O objetivo desta lista é elevar o nível de abstração. Todos os exercícios exigem a combinação de múltiplos fundamentos estudados até aqui. Organize seu raciocínio: prefira criar funções específicas para cada ação isolada (ex: uma função para cadastrar, outra para calcular) e utilize laços para manter os sistemas rodando até o usuário decidir sair voluntariamente.

1. Sistema de Gestão de Biblioteca

Crie um sistema para gerenciar uma livraria virtual. Inicie um array contendo 3 objetos, onde cada objeto representa um livro com as propriedades: **título**, **autor** e **disponível** (booleano, inicie todos como **true**).

Crie uma função **emprestarLivro(nomeDoLivro)** que receba uma string. A função deve buscar o livro no array utilizando um laço de repetição. Se encontrar e ele estiver **disponível**, altere essa propriedade para **false** e retorne "Empréstimo realizado com sucesso". Se não encontrar o título ou se ele já estiver emprestado, retorne uma mensagem de erro apropriada. No programa principal, crie um laço contínuo que exiba um menu ao usuário: 1 - Emprestar livro, 2 - Ver todos os livros (exibindo o título e a disponibilidade de cada um), 3 - Sair.

2. Gerenciamento de Frota de Táxis

Uma cooperativa precisa controlar sua frota. Crie um array contendo 2 objetos representando táxis. Cada táxi deve ter **placa**, **faturamentoDia** (inicie em zero) e um booleano **emServico** (inicie como falso).

Crie uma função **registrarCorrida(placa, valor)**. A função deve localizar o táxi no array. Se encontrar o veículo, simule a corrida mudando **emServico** para verdadeiro temporariamente e voltando para falso após o registro. Some o **valor** ao **faturamentoDia** do táxi correto e avise "Corrida registrada para [placa]". Crie um menu em looping para o motorista ou despachante poder registrar diversas corridas ao longo do dia para qualquer um dos veículos até decidir "Encerrar Expediente". Após o encerramento, utilize um laço para imprimir o faturamento final detalhado de cada placa.

3. Aplicativo de Finanças Pessoais com Categorias

Construa o objeto **Carteira** com as seguintes propriedades principais: **titular**, **saldoGeral** (inicie em zero) e **historicoDeTransacoes** (um array vazio).

Crie uma função isolada chamada **adicionarTransacao(tipo, categoria, valor)**. Esta função deve criar um novo objeto de transação (**{ tipoDaMovimentacao: "Receita" ou "Despesa", categoria: "Alimentação"/"Transporte"/"Lazer"/etc., montante: X }**) e adicioná-lo ao array histórico da carteira. Em seguida, a função deve calcular e atualizar o **saldoGeral** da carteira (somando se for receita, subtraindo se for despesa). No código principal, faça um menu iterativo onde o usuário possa: 1 - Lançar transação (escolhendo tipo, categoria e valor), 2 - Ver extrato completo (percorrendo o array com um laço), 3 - Ver resumo por categoria (somando os valores de cada categoria separadamente) e 4 - Sair. Ao sair, exiba o **saldoGeral** formatado na tela.

4. Sistema de Notas e Desempenho Escolar

Você possui um banco de dados representado por um array de objetos contendo três alunos. Cada aluno começa com a propriedade **nome** preenchida e uma propriedade **notas** sendo um array completamente vazio.

Crie um menu interativo contínuo onde o professor pode: 1 - **Registrar Nota**: Pede o nome do aluno, busca-o no array e insere uma nova nota numérica ao final do array de notas desse aluno. 2 - **Fechar o Semestre**: Ao acionar esta opção, chame uma função que varre todos os alunos. Para cada aluno na lista, calcule a média aritmética de suas respectivas notas usando um laço interno. Se a média for ≥ 7 , crie dinamicamente a propriedade **status: "Aprovado"**. Se for entre 5 e 6.9, **status: "Exame"**. Abaixo de 5, **"Reprovado"**. Imprima o boletim completo e formatado da turma toda.

5. Inventário Realista de RPG

Para um jogo de sobrevivência, crie um objeto **Personagem** com as propriedades: **nome**, **capacidadePesoMaximo** (ex: 50kg) e **listaInventario** (um array inicialmente vazio).

Desenvolva uma função **coletarLoot(nomeItem, pesoItem)**. A função deve obrigatoriamente iterar sobre a **listaInventario** para calcular a **soma exata** dos pesos de todos os itens que o personagem já está carregando. Se a soma do peso já ocupado somado ao **pesoItem** novo for menor ou igual à **capacidadePesoMaximo**, adicione o novo objeto (**{ nome: nomeItem, peso: pesoItem }**) ao array e retorne "Item guardado com segurança". Caso estoure a capacidade, recuse e retorne "Personagem sobrecarregado! O item ficou no chão". Crie um menu para o jogador encontrar itens aleatórios e tentar guardá-los até cansar da aventura.

6. Sistema Comercial de Reserva de Poltronas

Represente as fileiras Premium de um cinema criando um array contendo 5 objetos. Cada cadeira é um objeto com as chaves: `numeroAssento` (1 a 5), `ocupado` (false) e `nomeCliente` (null).

Construa uma função de reserva: `reservarAssento(numeroSelecionado, nomeComprador)`. Esta função deve procurar no array por um assento exato através do seu número usando um laço for embutido com if. Se o objeto daquela poltrona existir e estiver estritamente `ocupado = false`, mude `ocupado` para `true`, grave com o `nomeComprador` associado e avise "Ingresso emitido!". Caso já esteja como ocupado, avise de forma genérica que a transação não pôde ser completada pois o lugar está indisponível. Utilize um looping no arquivo inteiro (main), para simular a bilheteria continuamente perguntando o número da poltrona e quem quer sentar lá, permitindo visualizações do painel das poltronas vazias a todo momento.

7. Motor Inteligente de Descontos no Atacado

Um atacadista possui um estoque: crie um array contendo 4 objetos. Cada objeto possui `nomeProduto`, `categoria` ("Laticínios" ou "Diversos") e `precoCusto`.

Crie uma **arrow function** chamada `acionarLiquidacaoNoturna` que **não receba parâmetros**. A função deve acessar o array global diretamente. Utilizando um laço de repetição, percorra cada item do array e verifique: se a categoria for "Laticínios", crie uma nova propriedade `novoPrecoSugerido` com desconto de 30% sobre o `precoCusto`. Se a categoria for "Diversos", crie a mesma propriedade com desconto de apenas 5%. Após o laço, exiba no terminal todos os produtos com seus preços originais e os novos preços sugeridos.

8. Eleição em Código: Urna Digital Segura

Crie um array chamado `candidatos` contendo 3 objetos, cada um com `nome` e `quantidadeVotos` (inicie em zero).

Crie um laço `while` que simule uma sessão eleitoral. A cada rodada, exiba os candidatos numerados (ex: "1 - Fulano", "2 - Ciclano") e peça ao usuário para votar digitando o número correspondente, ou digitar "encerrar" para finalizar. Se o número for válido, incremente o `quantidadeVotos` do candidato correspondente. Se for inválido, exiba "Voto nulo".

Após o encerramento, crie uma **função separada** chamada `apurarVencedor` que receba o array de candidatos. A função deve percorrer todos os candidatos e, usando lógica condicional, determinar qual deles possui o maior número de votos. Retorne o nome do vencedor e sua quantidade de votos. Exiba o resultado final da eleição.

9. Painel de Agendamentos: Filtro por Especialidade

Uma clínica médica possui um array contendo 5 objetos. Cada objeto representa uma consulta agendada com as propriedades: `nomePaciente`, `horaMarcada` e `especialidade` (exemplos: "Ortopedia", "Clínica Geral", "Oftalmologia").

Crie uma função chamada `buscarPorEspecialidade(textoPesquisado)`. Dentro da função, crie um array vazio chamado `resultados`. Percorra o array de consultas com um laço: se a `especialidade` do objeto atual for igual ao texto pesquisado, insira esse objeto no array `resultados`. Ao final do laço, retorne o array `resultados`. No programa principal, crie um menu contínuo que pergunte ao recepcionista "Qual especialidade deseja filtrar?". Chame a função com a resposta, receba o retorno e exiba as consultas encontradas de forma organizada no terminal. Permita que o recepcionista faça múltiplas buscas até escolher sair.

10. Desafio Máster: Caixa Eletrônico ATM

Crie dois objetos que representam o sistema:

1. `CaixaEletronico` — contendo a propriedade `gavetas`, que é um array de notas disponíveis: `[ { valor: 100, quantidade: 10 }, { valor: 50, quantidade: 10 }, { valor: 20, quantidade: 15 } ]`.
2. `Cliente` — contendo `nome` e `saldoEmConta` (inicie em R\$ 800,00).

Crie um menu com `while` onde o cliente possa solicitar saques. Ao receber o valor desejado, o sistema deve:

1. **Verificar o saldo:** Se o valor do saque for maior que o `saldoEmConta`, recuse a operação.
2. **Montar as notas:** Percorra o array de `gavetas` da maior nota para a menor. Para cada gaveta, calcule quantas notas daquele valor cabem no saque restante (sem ultrapassar a quantidade disponível na gaveta). Subtraia do valor restante e continue para a próxima gaveta.
3. **Validar a operação:** Se após percorrer todas as gavetas o valor restante for zero, o saque é possível — subtraia o valor do saldo do cliente e atualize a quantidade de notas em cada gaveta. Se o valor restante for diferente de zero (ex: o cliente pediu R\$ 35,00), **recuse a operação inteira** sem alterar nem o saldo, nem as gavetas.

Ao sair do menu, exiba o saldo final do cliente e o estado atualizado de todas as gavetas para conferência gerencial.

Dica: Lembrem-se que os dados capturados pelo `prompt-sync` vêm como formato de Texto (String). Para realizar cálculos matemáticos ou aplicar regras lógicas de maior/menor,

é essencial converter essas entradas para Número (usando `Number()` , `parseInt()` ou `parseFloat()`) antes de usá-las.

LionsDev • Professor Nicolas Cardoso Motta

Exercícios dos Fundamentos da Programação - Módulo 02